



Auto-generating Code

Using tools to auto-generate code from high-level models or specifications looks like a cool idea. Though generating code instead of 'hand-writing' it from scratch has many advantages, one needs to also be aware of the disadvantages of this approach. This article takes a closer look at auto-generating code.

When I first got exposed to auto-code generators, I was quite impressed with what they could achieve. For example, YACC (Yet Another Compiler Compiler) is a compiler generator. To explain what a 'compiler generator' means in layman terms, it's when you provide the specification (i.e., language grammar) for parsing interspersed with hand-coded actions, and the YACC tool generates code corresponding to that grammar. YACC is a 'compiler compiler'; in other words, it's a compiler that takes the grammar and generates as output the code for a compiler that recognises that grammar. So, instead of writing a compiler (the parser component of a compiler, to be specific), which can be a *really* complex task, you can specify the grammar in the format required by YACC, and you get parser code 'auto-generated'. You can use that parser to parse the input source files.

Similarly, you can generate a 'lexical analyser' using the Lex tool. A lexical analyser takes source code, which is a stream of characters, and generates a stream of tokens (for this reason, a lexical analyser is also known as a *tokeniser*); these tokens are then fed to the parser. Coming back to Lex, you can specify what tokens you need to generate in the format required by Lex, and the Lex tool will generate a lexical analyser code for you. Looks cool, doesn't it?

Tools like lex and yacc can be considered to be 'domain-specific languages' (DSLs). These are designed for a specific domain, and are specialised languages for performing tasks in those domains. You can contrast DSLs with general-purpose languages such as C++ and Java, which are pretty much domain-neutral, and you can use them for a vast range of programming tasks. Not all DSLs generate compilable code. For example, SQL (Structured Query Language) is a DSL and is meant for interacting with a database. These DSLs can, in turn, be considered as Application Oriented Languages (AOLs). AOLs are specialised languages that are designed to program for a specific application or problem domain. AOLs allow us to worry about the problem domain, and help us focus on specifying the problem instead of the implementation details. AOLs are a good target for

auto-generating code. Well-known examples are code generators from UML diagrams—where you specify your high-level design in UML diagramming tools (such as ArgoUML, Enterprise Architect and Rational Rose), and you can get code automatically generated from those diagrams. Other examples are MatLab and Simulink, used for mathematical analysis and modelling.

Another well-known kind of auto-code generator includes GUI builders, which most of you may already be familiar with. For example, you can use tools such as NetBeans (for Java) or Visual Studio (for C#/VB) for UI design, and these tools will automatically generate compilable source code. Of course, you need to add things such as event-handling code and business-logic code to the generated code, but they already allow you to focus on the UI design by just dragging and dropping UI controls instead of writing all the template code in a text editor.

At some point you may have used 'wizards' that generate code for you for your specific needs. These are quite popular, especially for simplifying programming for repetitive tasks. They are widely used, especially in enterprise projects, since they help software companies make their programmers more productive. Also, product companies that sell software packages targeting programmers, attract potential developers by showing how software developers can reduce their programming efforts if they buy their software package. As a programmer, you can now simply click or drag and drop to create programs instead of typing at the keyboard for long hours. Looks quite attractive, doesn't it?

So, what are the advantages of 'generating code'?

1. **Improved productivity.** Automated code generators allow you to work at the level of specifications or models, and communicate in the terms used in the application domain. With general-purpose languages, you need to work in terms of low-level constructs such as *for* and *while* loops, which is quite tedious and time consuming.
2. **Reduced complexity.** When programmers work at higher levels of abstraction, they can perform complex tasks with ease, compared to low-level constructs in general-purpose languages.